Imagine you're sitting in an Interview for an SDE-II role at Amazon, and you're asked this question: "Design a distributed logging system."

I'm an Ex-Amazon backend engineer with over 7 years of experience...Here's how I would break it down if I were the one being interviewed:

The real question here is not "which tool would you pick". It's whether you can design a high-throughput, highly reliable ingestion plus storage platform that stays alive during incidents, and still lets engineers debug fast.

There are lots of aspects we can cover here, but I'll only cover the high-level design.

1. Requirements

The classic pain in distributed systems:
- Debugging is slow because logs are spread across machines
- Visibility is poor because you cannot see the whole system in one view
- Logging is inconsistent because formats and timestamps don't match

Functional requirements
- Unified log collection from all services and environments
- Fast search and retrieval by service, time range, keywords
- Trace a single request end-to-end using a correlation ID
- Central dashboards for monitoring and alerting
- Retention and archival (hot vs cold) with policy-based purge

Non-functional requirements
- Low latency and minimally intrusive (logging must not slow down user requests)
- Horizontal scalability (add machines when volume grows)
- High availability and reliability (logs should not disappear during outages)
- Security and access control (logs can contain sensitive identifiers)

If you want numbers to anchor the discussion, I usually assume something like:
- Ingestion: hundreds of thousands of logs per second
- Hot storage growth: multiple TB per day
- Search load: hundreds of queries per second
- Availability target: five 9s is a great bar for "observability backbone"

2. High-level architecture

I split the system into a clean pipeline:

- Log producers (microservices)
- Log agents on every node (collection + masking + enrichment)

- Ingestion buffer (Kafka)
- Stream processing (Flink)
- Storage backends (search, metrics, archive)
- Query and dashboards (for humans and alerts)

3. Correlation ID at the edge

First thing I want in the design: A correlation ID that is attached to the request at the entry point and flows through every service.

That single ID turns “random logs everywhere” into a timeline you can replay.

4. Log agents on every machine

Think of them as the sidecar that does the boring but critical work:
- Tail logs locally
- Add metadata: service name, env, region, instance, version
- Mask obvious sensitive fields (still, never log secrets)
- Batch and ship asynchronously so the app never blocks

Common choices are Fluent Bit or Filebeat, but the tool name is not the point. The behavior is.

5. Kafka as the ingestion backbone

Kafka is doing three jobs here:

- Buffer spikes so downstream systems don’t fall over
- Decouple producers from consumers
- Make the pipeline recoverable using offsets (replay is life-saving after failures)

Partitioning strategy usually comes up in interviews.
My default:
Partition by something that spreads load evenly, often service-name plus shard, not by correlation ID (correlation IDs can hotspot during incidents).

6. Flink for real-time processing

Flink is where you turn raw logs into something usable:

- Parse and validate schema (structured logs win)
- Enrich with more context
- Filter noise (debug spam in prod burns money)
- Aggregate into metrics (error rate, latency percentiles, top error codes)
- Route to the right storage systems

This is also where you apply backpressure properly so you don't kill the pipeline under load.

7. Storage backends

I keep it simple and purpose-driven:

Search store (OpenSearch / Elasticsearch)

- For interactive log search
- Indexed for fast queries by time, service, correlation ID, keywords

Metrics store (TimescaleDB / Prometheus family)

- For time-series charts and alerting
- Works best when you store aggregates, not raw logs

Archive store (S3)

- Cheap, durable long-term storage
- Store compressed columnar files (Parquet) for offline queries and audits

8. How engineers actually use it

- Kibana or OpenSearch Dashboards on top of the search store for "what happened"
- Grafana on top of the metrics store for "how bad is it, and is it getting worse"
- Athena on top of S3 for "show me the last 6 months"

9. A few interview follow-ups I always prepare for

If the interviewer wants to go deeper, these are the "senior signals":

- Exactly-once vs at-least-once ingestion, and how you handle duplicates
- Ordering guarantees (and why you usually don't need total ordering)
- Schema evolution (versioned events, Protobuf/Avro, backward compatibility)
- Hot vs warm vs cold retention strategy
- Multi-region availability and disaster recovery
- Cost controls (sampling, filtering, tiered storage)

That's the skeleton.

If you can explain this calmly, with tradeoffs, and a couple of concrete flows, you're not just "designing a logging system".

You’re showing you understand how modern production systems stay observable when everything else breaks.